

CS149 Assignment 1: Performance Analysis

Name: _____ SUNet ID: _____

Test machine used for all measurements below. The handout targets a *quad-core* Intel i7 (4 cores / 8 threads, 8-wide AVX2). All numbers here were collected on:

AMD Ryzen 7 6800HS - 8 physical cores / 16 threads (SMT2), AVX2 + FMA (no AVX-512), Ubuntu 24.04 (WSL2), g++ 13.3 -03 , ISPC 1.28.1 (--target=avx2-i32x8). Because this box has *twice* the cores of the reference machine, the core-scaling ceilings are roughly doubled (e.g. Program 3 theoretical max $\approx 8 \text{ lanes} \times 8 \text{ cores} = \mathbf{64\times}$, vs $32\times$ on a quad-core). Trends and conclusions are identical; only the absolute ceilings differ.

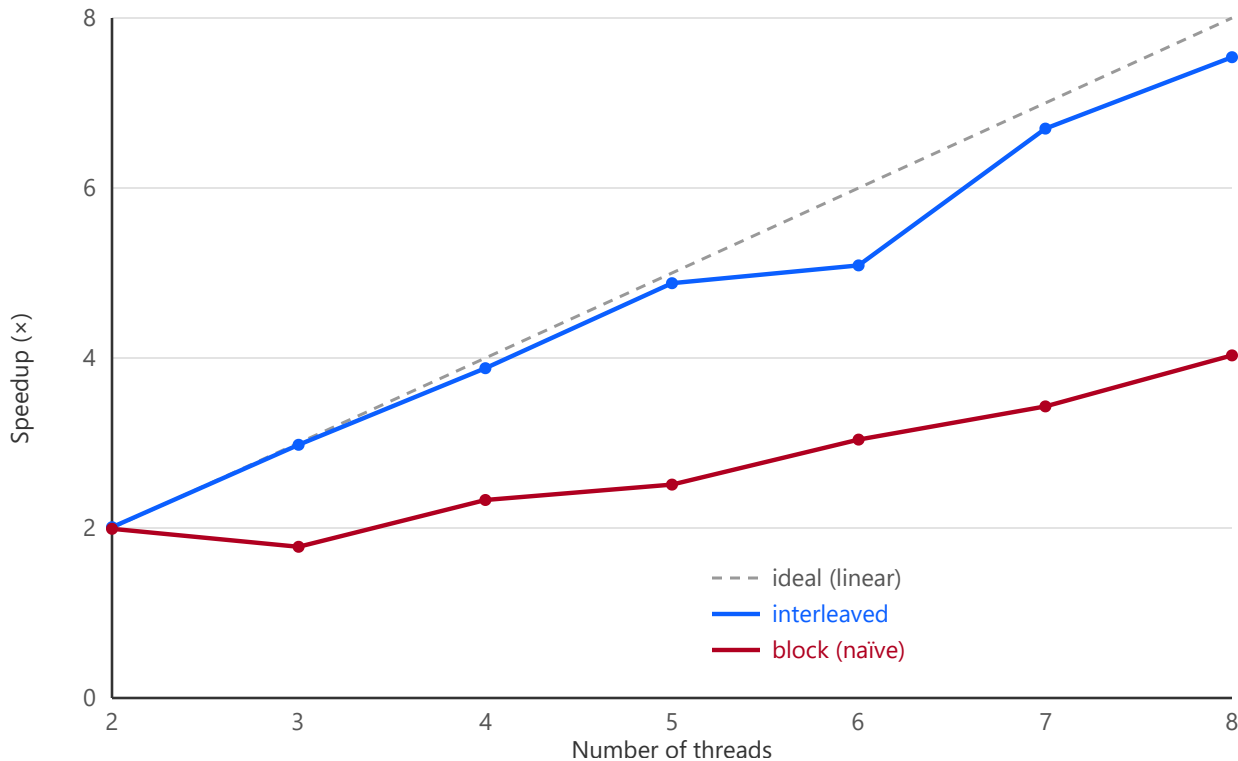
Program 1 - Parallel Fractal Generation Using Threads

Approach. Each worker calls `mandelbrotSerial()` on a subset of image rows. I first implemented the naïve *block* decomposition (thread *t* gets a contiguous band of `height/numThreads` rows), then replaced it with an *interleaved* (cyclic) decomposition that fixes load imbalance:

```
// Final: interleaved rows - thread t computes rows t, t+numThreads, t+2*numThreads, ...
for (unsigned int row = args->threadId; row < args->height; row += args->numThreads)
    mandelbrotSerial(x0,y0,x1,y1, width, height, /*startRow=*/row, /*numRows=*/1, maxIterations, output);
```

Speedup vs. thread count (View 1)

Threads	2	3	4	5	6	7	8	16
Block (naïve)	1.99×	1.78×	2.33×	2.51×	3.04×	3.43×	4.03×	7.44×
Interleaved (final)	2.01×	2.98×	3.88×	4.88×	5.09×	6.70×	7.54×	10.22×



Speedup vs. thread count for View 1. Note the block curve *dips at 3 threads*; the interleaved curve tracks the ideal line closely.

Q: Is the speedup linear? Explain.

No, the **block** decomposition is far from linear and is even *non-monotonic* - 3 threads (1.78x) is *slower* than 2 threads (1.99x). The Mandelbrot workload is spatially skewed: rows near the vertical center of the image are inside/near the set and cost ~ `maxIterations` per pixel, while rows at the top and bottom escape almost immediately. With 2 threads the top and bottom halves happen to be roughly symmetric, so the split is balanced (~2x). With 3 threads the entire heavy center band lands on the middle thread, which becomes a bottleneck. The **interleaved** decomposition removes this skew and tracks the ideal line closely (7.54x on 8 threads).

Q: Per-thread timing (block decomposition, 8 threads, View 1).

Thread	0	1	2	3	4	5	6	7
Rows	0-149	150-299	300-449	450-599	600-749	750-899	900-1049	1050-1199
Time (ms)	5.9	32	63	96	96	65	32	7.0

The center threads (3, 4) do ~**16x** the work of the edge threads (0, 7). Since the phase finishes only when the *slowest* thread finishes (~96 ms), total speedup is pinned near 4x no matter how many threads are added - exactly what the graph shows. This directly explains the sub-linear block curve.

Q: Describe how you modified the code to improve speedup, and your final 8-thread speedup.

I switched from contiguous bands to an **interleaved (cyclic) row assignment**: thread t computes rows $t, t+P, t+2P, \dots$. Because expensive and cheap rows are dealt out round-robin, every thread receives a near-identical mix of work, so per-thread times converge and the phase is no longer bottlenecked by one thread. Final **8-thread speedup: 7.54x (View 1), 7.27x (View 2)** - within the target 7-8x band.

Q: What happens with 16 threads? Is there an additional 2x over 8?

No - 16 threads gives only **10.2x** (View 1), not ~15x. The machine has **8 physical cores**; threads 9-16 run on the second SMT/hyperthread of each core and share the same execution units, L1/L2, and FP pipelines. SMT hides latency but adds little throughput for this compute-bound, ALU-heavy kernel, so the extra 8 threads yield only a modest ~1.35x on top of 8. (On the handout's quad-core, the analogous crossover is at 4→8 threads.)

Program 2 - Vectorizing Code Using SIMD Intrinsics

Approach (`clampedExpVector`). Process `VECTOR_WIDTH` elements per iteration. Each lane starts `result = 1.0` and multiplies by `x` exactly `y` times using a running mask `(y > 0)`; lanes whose exponent is exhausted are masked off while the rest continue. A final masked compare clamps values `> 9.999999`. The ragged final block is handled with `_cs149_init_ones(N - i)` so the solution is correct for any `N` (verified at `N = 17, 103, 10000`).

Vector utilization vs. `VECTOR_WIDTH` (`-s 10000`)

<code>VECTOR_WIDTH</code>	2	4	8	16
Vector utilization	90.1%	85.8%	83.7%	82.6%
Total vector instructions	203,143	118,213	64,533	33,708

Q: Does utilization increase, decrease, or stay the same as width changes? Why?

It decreases as the vector grows wider. The multiply loop runs until the lane with the *largest* exponent in the block is finished; every other lane sits masked-off for the remaining iterations (this is SIMD divergence). A wider block is statistically more likely to contain a large-exponent element that drags out the whole gang, so the fraction of wasted lanes grows. The instruction count roughly halves each time the width doubles (fewer blocks), but the *utilization* - useful lane-work ÷ total lane-work - slips downward.

Extra credit - `arraySumVector` in $O(N/VECTOR_WIDTH + \log_2(VECTOR_WIDTH))$.

Implemented and passing. Phase 1 accumulates all blocks into one wide register with N/W vector adds. Phase 2 tree-reduces the W partial sums to a scalar in $\log_2 W$ rounds, each round doing one `_cs149_hadd_float` (sum adjacent pairs) followed by one `_cs149_interleave_float` (pack the pair-sums into the front half). After $\log_2 W$ rounds every lane holds the total.

Program 3 - Parallel Fractal Generation Using ISPC

Part 1 - ISPC SIMD (no tasks)

Q: What is the maximum theoretical speedup from 8-wide AVX2? Why is the observed speedup lower?

The gang is 8-wide, so the ceiling from SIMD alone is **8x**. Measured: **View 1 = 6.05x**, **View 2 = 5.14x**. The gap is SIMD divergence: the 8 pixels in a gang have *different* iteration counts (some escape after a few iterations, some run to `maxIterations`), but the gang can only retire when its *slowest* lane finishes, so lanes that already escaped are wasted. The loss is worst along the fractal boundary and inside the black interior, where neighbouring pixels diverge sharply. View 2 is more divergent overall, giving the lower speedup. Fixed overheads (loop setup, memory) shave off a little more.

Part 2 - ISPC with tasks

Approach. The starter launches just 2 tasks (`rowsPerTask = height/2`), which uses only ~2 cores and inherits the same center-heavy imbalance. I launch **many fine-grained tasks** (`ROWS_PER_TASK = 6` → ~134 tasks for height 800) and clamp the last task's row range. Small tasks let the ISPC runtime hand each idle core the next strip on demand (dynamic load balancing).

rows/task	≈ tasks	View 1	View 2
height/2 (starter)	2	11.6×	8.3×
1	800	22.3×	14.0×
2	400	54.7×	27.1×
4	200	59.9×	45.1×
6 (chosen)	134	64.6× (run: 58.6×)	51.1× (run: 47.9×)
16	50	63.2×	44.1×
25	32	65.1×	37.5×

Chosen config easily exceeds the 32× target (58-65× View 1, 48-51× View 2 on this 8-core box; >32× is what a quad-core would reach). Methodology: sweep granularity. Too coarse (2 tasks) → only 2 cores busy and load-imbalanced; too fine (1 row/task, 800 tasks) → per-launch overhead dominates and speedup collapses to 22×. The sweet spot is enough tasks to keep all 8 cores/16 threads balanced (~130) while staying coarse enough to amortize launch cost - `ROWS_PER_TASK = 6`.

Extra credit - `std::thread` (Prog 1) vs. ISPC tasks: what happens if you launch 10,000 of each?

Program 1 spawns one *OS thread* per unit. 10,000 `std::thread`s = 10,000 kernel threads on 8 cores → massive oversubscription: context-switch storms, scheduler overhead, and ~10 MB× stacks of memory pressure; performance craters. An ISPC `launch[10000]` creates 10,000 lightweight *tasks* placed in a work queue and drained by a small pool of exactly `#cores` worker threads. Tasks are cheap descriptors, not kernel threads, so 10,000 of them just gives the runtime fine-grained work to load-balance - near-ideal behaviour. Tasks are an abstraction over a thread pool; raw threads are the scheduled resource itself.

Program 4 - Iterative Square Root

Input	ISPC (SIMD only)	ISPC + tasks
Baseline (random [0.001, 2.999])	5.01×	61.1×
Best case (all = 2.998)	6.60×	69.5×
Worst case (1 hard value / gang of 8)	0.90×	8.6×

Q: Report the SIMD speedup and the multi-core speedup, and quantify each contribution.

On the baseline input, **SIMD alone gives ~5.0×** and **SIMD + tasks gives ~61×**. So vectorization contributes ~5×, and multi-core parallelism contributes an additional ~12× on top of that (61 ÷ 5), using 64 tasks spread over 8 cores + SMT. The two axes are roughly multiplicative.

Q: Modify the input to maximize the ISPC (no-task) speedup. Explain.

Set **every element to 2.998** - the value that takes the most Newton iterations to converge (note $x = 3.0$ never converges: the update collapses the guess to 0). Two effects combine: (1) all 8 lanes run the *same* iteration count → zero divergence, full lane utilization; (2) many iterations per element → the kernel is compute-bound, so per-element and memory overheads are amortized. Result: **6.60×**, close to the 8× ceiling (the residual gap is memory traffic for 20M floats plus fixed loop overhead).

Q: Construct an input that *minimizes* the ISPC (no-task) speedup. Why is 8-wide SIMD inefficient here?

Make **one element in every gang of 8 the hard value (2.998) and the other seven trivial (1.0, which converges in zero iterations)**. The gang still loops to the hard lane's iteration count while the other seven lanes are masked-off the whole time - so the vector unit does

the work of a single scalar element per 8 lanes. Result: **0.90×**, i.e. *slower* than scalar, because we pay masking/vector overhead for essentially no parallel work. This is the canonical demonstration that divergence destroys SIMD efficiency.

Extra credit - hand-written AVX2 intrinsics.

Not submitted. The path: load 8 floats with `_mm256_loadu_ps`, run the Newton update with `_mm256_fmadd_ps / _mm256_mul_ps`, compute the mask with `_mm256_cmp_ps` (`|guess2·x - 1| > threshold`), and blend updates via `_mm256_blendv_ps`, looping while the mask is non-zero (`_mm256_movemask_ps`). That mirrors exactly what ISPC generates for `--target=avx2-i32x8`, so it should match ISPC's ~5× on the baseline input.

Program 5 - BLAS saxpy

Version	Time	Bandwidth
ISPC (no tasks)	10.85 ms	27.5 GB/s
ISPC + tasks	8.01 ms	37.2 GB/s
Speedup from tasks		1.36×

Q: What speedup do tasks give? Is a large improvement possible? Why or why not?

Only **1.36×**. saxpy (`result = scale·X + Y`) does 2 loads + 1 store and a single FMA per element - an arithmetic intensity of ~0.08 flop/byte. It is **memory-bandwidth bound**, not compute bound. One or two cores already saturate the DRAM controller, so extra cores/threads add almost nothing. No substantial speedup is achievable on this platform without reducing memory traffic.

Extra credit - why is the bandwidth computed as `4·N·sizeof(float)` when only 3 arrays are touched?

Write-allocate. Storing to `result[]` first triggers a read-for-ownership: the destination cache line must be fetched from memory before it can be (over)written. So the "one store" actually costs one read + one write. Total traffic = load X + load Y + RFO(result) + write(result) = **4·N·sizeof(float)**. Using non-temporal / streaming stores (e.g. `_mm256_stream_ps`) skips the RFO, cutting traffic to 3·N and giving up to a $4/3 \approx 1.33\times$ bandwidth improvement - the realistic "best case" for this kernel.

Program 6 - K-Means Clustering

Measured on a locally-generated dataset ($M = 200,000$, $N = 100$, $K = 3$, $\epsilon = 0.1$, 51 iterations to convergence), since the official `data.dat` lives on Stanford AFS. The relative speedups are data-independent.

Profiling methodology. I wrapped each of the three phase calls in the main loop with `CycleTimer` and summed per-phase time across all iterations:

Phase	Time	Share	Complexity
computeAssignments	2529 ms	68%	$O(K·M·N)$
computeCost	721 ms	19%	$O(M·N)$
computeCentroids	455 ms	12%	$O(M·N)$
Total (baseline)	3705 ms	100%	

Q: What is the bottleneck, and how did you optimize it?

Hypothesis: `computeAssignments` is the hotspot - it is the only $O(K·M·N)$ phase and calls the expensive `dist` (a `pow` + `sqrt` over N dimensions) for every (point, centroid) pair. Profiling confirms it at 68%. **Fix:** parallelize *only* this phase across 8 `std::threads`,

partitioned over *data points*. Because each point is written by exactly one thread, the ranges are disjoint - **no locks, no races**. I also swapped the loop nesting to point-outer so each point keeps its running minimum in a register (dropping the M-sized `minDist` scratch array).

Q: Final speedup.

`computeAssignments` fell from **2529** → **384 ms (6.6× on the phase)**; total fell from **3705** → **1473 ms = 2.52× overall** (target ≥ 2.1×). The clustering output (`end.log`) is byte-identical to the baseline - the result is unchanged.

Q: There are different axes of parallelism - how did you decide?

Two axes exist: over *points* (M, ~200k-way) and over *centroids* (K, only 3-way). I chose points because $M \gg K$ gives far more parallel slack and perfectly balanced, contention-free partitions; a K-way split would cap at 3× and be imbalanced. I spent the effort on the single phase Amdahl's law rewards most (the 68% one). After the fix, `computeCost` + `computeCentroids` now dominate (~77% of the remaining 1.5 s), so parallelizing `computeAssignments` alone is capped near $1/0.32 \approx 3.1\times$ overall - consistent with the measured 2.5×.

Deliverable code files: Program 2 → `prob2.cpp` ; Program 6 → `prob6.cpp` . Additional modified sources (Programs 1, 3, 4) and the data generator are included under `code/` for reproducibility. All code compiles and runs unmodified with ISPC 1.28.1 + g++.